

Ques.

127. Word Ladder

Hard Topics Companies

A **transformation sequence** from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the **number of words in the shortest transformation sequence** from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

Input: `beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`

Output: 5

Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

- 1) Put each element of wordlist in the set data

Start from 'hit', change one letter and check if it's exist in set or not, if exist try adding it in the queue ^{along with its no}; now delete the word 'hit' now, similar process for next words.

```
1 class Solution {
2 public:
3     int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
4         set<string> st(wordList.begin(), wordList.end());
5         queue<pair<string,int>> q;
6         q.push({beginWord,1});
7         st.erase(beginWord);
8         while(!q.empty()){
9             string word = q.front().first;
10            int steps = q.front().second;
11            q.pop();
12            if(word==endWord) return steps;
13            for(int i = 0; i<word.size(); i++){
14                char temp = word[i];
15                for(char ch = 'a'; ch<='z'; ch++){
16                    word[i] = ch;
17                    if(st.find(word)!=st.end()){
18                        q.push({word,steps+1});
19                        st.erase(word);
20                    }
21                }
22                word[i] = temp; // restored the original letter of that word before changing other letters.
23            }
24        }
25        return 0;
26    }
27 };
```

Hard Topics Companies

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `s_k == endWord`

Example 1:

```

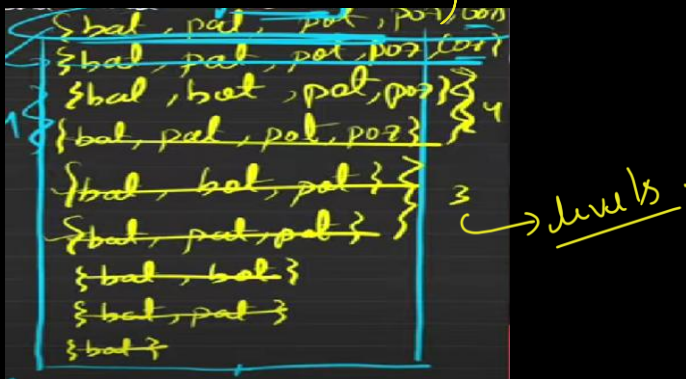
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]
Explanation: There are 2 shortest transformation sequences:
"hit" -> "hot" -> "dot" -> "dog" -> "cog"

```

Ex.

```
graph TD
    dot[dot] --> dog[dog]
    dot --> dot2[dot.]
```

(don't delete 'dot' just now) (delete the 'dot' when it's all level
are done).



```

1  class Solution {
2  public:
3      vector<vector<string>> findLadders(string beginWord, string endWord, vector<string>& wordList) {
4          unordered_set<string> st(wordList.begin(), wordList.end());
5          queue<vector<string>> q;
6          vector<string> usedonlevel;
7          q.push({beginWord});
8          usedonlevel.push_back(beginWord);
9          int level = 0;
10         vector<vector<string>> ans;
11         while(!q.empty()){
12             vector<string> vec = q.front();
13             q.pop();
14             if(vec.size() > level){
15                 level++;
16                 for(auto it: usedonlevel){
17                     st.erase(it);
18                 }
19                 usedonlevel.clear();
20             }
21             string word = vec.back();
22             if(word == endWord){
23                 if(ans.size() == 0){
24                     ans.push_back(vec);
25                 }
26                 else if(ans[0].size() == vec.size()){
27                     ans.push_back(vec);
28                 }
29             }
30             for(int i = 0; i < word.size(); i++){
31                 char original = word[i];
32                 for(char ch = 'a'; ch <= 'z'; ch++){
33                     word[i] = ch;
34                     if(st.find(word) != st.end()){
35                         vec.push_back(word);
36                         q.push(vec);
37                         usedonlevel.push_back(word); // mark as visited on the usedonlevel. to make it deleted
38                     }
39                     vec.pop_back(); // means if bat converted to bot add in the sequence, and when bat
40                                     converted to pat remove the bot and then add pat to form a new sequence.
41                     word[i] = original;
42                 }
43             }
44             return ans;
45         }
46     }

```

Dijkstra Algorithm:- It can be implemented using prio-queue, set.

Method:-

- We will declare distance array and queue({dis, node})
- Initially in distance array value of distance for each node is ∞ whenever lesser value comes, put that into the queue and calculate distance of adjoint nodes, whenever a distance of a node is updated put that in the queue.

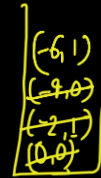
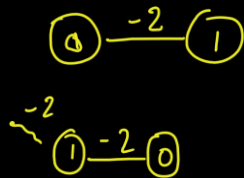
2) Using Set - As in the priority queue we were trying to keep smallest distance at the top, but in the set data structure it by default stores the smallest element at the top. So, set logic is same.

→ Using set makes one more difference - Suppose we have $(10, 5)$ in the set, but while processing another node $(weight, edge)$ we get $(8, 5)$. So we will be able to delete the $(10, 5)$.

To access the top element of a set, use -
`auto it = *st.begin();`

→ Dijkstra Algorithm cannot be used if there is single edge having negative weight.

Reason-



as -9 is lower than 0 $\rightarrow$ (the distance to node 0) so, dis. of node-0 will be updated to -9 and this will be continued infinitely.

Time-complexity for Dijkstra Algo = $E \log V$

$$\begin{aligned}
 &O(V \times (\text{pop vertex from min heap} + \text{no. of edges on each vertex} \times \text{push into PQ})) \\
 &O(V \times (\log(\text{heap size}) + ne \times \log(\text{heap size})) \\
 &O(V \times (\log(\text{heap size}) \times (ne + 1))) \quad \text{one vertex can have } (V-1) \text{ edges.} \\
 &O(V \times V \times \log(\text{heap size})) \\
 &O(V^2 \times \log(\text{heap size})) \quad \text{every time compresses \& gets all the nodes in} \\
 &O(V^2 \times \log(V^2)) \\
 &O(V^2 \times 2 \log V) \quad E = V^2
 \end{aligned}$$

Ques - Print shortest Path -

We will use Dijkstra algorithm & along with that we will use parent array. Suppose we are getting node 5 from node 3, then at the index 5 we will store 3.

Ques. Shortest distance in a Binary Maze.

Shortest Distance in a Binary Maze



Difficulty: **Medium**

Accuracy: **58.22%**

Submissions: **79K+**

Points: **4**

Average Time: **20m**

Given a $n * m$ matrix **grid** where each element can either be **0** or **1**. You need to find the shortest distance between a given source cell to a destination cell. The path can only be created out of a cell if its value is 1.

If the path is not possible between source cell and destination cell, then return **-1**.

Note : You can move into an adjacent cell if that adjacent cell is filled with element 1. Two cells are adjacent if they share a side. In other words, you can move in one of the four directions, Up, Down, Left and Right. The source and destination cell are based on the zero based indexing. The destination cell should be 1.

Example 1:

Input:

```
grid[][] = {{1, 1, 1, 1},
             {1, 1, 0, 1},
             {1, 1, 1, 1},
             {1, 1, 0, 0},
```

Solution - Here we will use dijkstra algorithm, in which priority queue will store {distance, {row, col}} and we will use distance matrix.

→ Here we can also use queue, as the adjacent distance is always 1, so, shortest distance will be at top always adjacent.

```

5  int shortestPath(vector<vector<int>> &grid, pair<int, int> source,
6      pair<int, int> destination) {
7      // code here
8      int n = grid.size();
9      int m = grid[0].size();
10     vector<vector<int>>dis(n,vector<int>(m,INT_MAX));
11     int source_row = source.first;
12     int source_col = source.second;
13     int dest_row = destination.first;
14     int dest_col = destination.second;
15     dis[source_row][source_col] = 0;
16     queue<pair<int,pair<int,int>>>q;
17     q.push({0,{source_row,source_col}});
18     int del_row[] = {0,0,-1,1};
19     int del_col[] = {-1,1,0,0};
20     while(!q.empty()){
21         int rw = q.front().second.first;
22         int col = q.front().second.second;
23         int wt = q.front().first;
24         q.pop();
25         if(wt>dis[rw][col])continue;
26         for(int i = 0; i<4; i++){
27             int newrow = rw+del_row[i];
28             int newcol = col+del_col[i];
29             if(newrow>=0 && newrow<n && newcol>=0 && newcol<m && grid[newrow][newcol]==1){
30                 if(dis[newrow][newcol]> wt+1){
31                     dis[newrow][newcol] = wt+1;
32                     q.push({dis[newrow][newcol],{newrow,newcol}});
33
34
35     }
36 }
37 if(dis[dest_row][dest_col]==INT_MAX)return -1;
38 return dis[dest_row][dest_col];
39
40 }
41 };
42

```

Ques- Path with minimum Effort -

1631. Path With Minimum Effort

Medium

Topics

Companies

Hint

You are a hiker preparing for an upcoming hike. You are given `heights`, a 2D array of size `rows x columns`, where `heights[row][col]` represents the height of cell `(row, col)`. You are situated in the top-left cell, `(0, 0)`, and you hope to travel to the bottom-right cell, `(rows-1, columns-1)` (i.e., **0-indexed**). You can move **up, down, left, or right**, and you wish to find a route that requires the minimum **effort**.

A route's **effort** is the **maximum absolute difference** in heights between two consecutive cells of the route.

Return the minimum **effort** required to travel from the top-left cell to the bottom-right cell.

1	2	2
3	8	2
5	3	5

Input: heights = [[1,2,2],[3,8,2],[5,3,5]]

Output: 2

Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells. This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.

Ans. We will use dijkstra algorithm for it, in which in the priority queue we will store {effort, {row, col}}, in the distance matrix we will store min_effort till each row-col. which will be $\min(\text{dis}[\text{row}][\text{col}], \max(\text{effort of prev row-col}, \text{abs}(\text{prev row-col-val} - \text{current-row-col-val})))$.

```

3 int minimumEffortPath(vector<vector<int>>& heights) {
4     int n = heights.size();
5     int m = heights[0].size();
6     vector<vector<int>>dis(n, vector<int>(m, INT_MAX));
7     priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<pair<int, pair<int, int>>>> pq;
8     pq.push({0, {0, 0}});
9     dis[0][0] = 0;
10    int del_row[] = {0, 0, -1, 1};
11    int del_col[] = {-1, 1, 0, 0};
12    while(!pq.empty()){
13        int row = pq.top().second.first;
14        int col = pq.top().second.second;
15        int effort = pq.top().first;
16        pq.pop();
17        if(row==n-1 && col==m-1) return effort; // as priority_queue stores with min effort at top so the first we get heights[n-1][m-1] that will be the minimum
18        // we can also remove this line and return dis[n-1][m-1] in the last.
19        if(effort > dis[row][col]) continue;
20        for(int i = 0; i < 4; i++){
21            int adjrow = row + del_row[i];
22            int adjcol = col + del_col[i];
23            if(adjrow >= 0 && adjrow < n && adjcol >= 0 && adjcol < m){
24                int adjeffort = abs(heights[row][col] - heights[adjrow][adjcol]);
25                if(dis[adjrow][adjcol] > max(effort, adjeffort)){
26                    dis[adjrow][adjcol] = max(effort, adjeffort);
27                    pq.push({dis[adjrow][adjcol], {adjrow, adjcol}});
28                }
29            }
30        }
31    }
32    return 0;
}

```

Ques-

787. Cheapest Flights Within K Stops

Solved

Medium Topics Companies

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return **the cheapest price from `src` to `dst` with at most `k` stops**. If there is no such route, return `-1`.

We will use dijkstra with queue storing {stops, {node, dist}} we need to sort condition on stops only once.
i.e. if(stops > k) continue.

```

1 class Solution {
2 public:
3     int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
4         vector<vector<pair<int,int>>>adj(n);
5         for(int i = 0; i<flights.size(); i++){
6             adj[flights[i][0]].push_back({flights[i][1],flights[i][2]});
7         }
8         queue<pair<int,pair<int,int>>>q;
9         q.push({0,{src,0}});
10        vector<int>dis(n,INT_MAX);
11        dis[src] = 0;
12        while(!q.empty()){
13            int node = q.front().second.first;
14            int stop = q.front().first;
15            int distance = q.front().second.second;
16            q.pop();
17            if(stop>k)continue;
18            for(auto it: adj[node]){
19                int adjnode = it.first;
20                int adjstop = stop + 1;
21                int adjdistance = distance + it.second;
22                if(dis[adjnode]>adjdistance){
23                    dis[adjnode] = adjdistance;
24                    q.push({adjstop,{adjnode,dis[adjnode]}});
25                }
26            }
27        }
28        if(dis[dst]==INT_MAX)return -1;
29        else return dis[dst];
30    }
31 };

```

Ques. Minimum Multiplication to reach the end-

Minimum Multiplications to reach End



Difficulty: **Medium**

Accuracy: **48.94%**

Submissions: **169K+**

Points:

4 Average Time: **20m**

Given **start**, **end** and an array **arr** of **n** numbers. At each step, **start** is multiplied with any number in the array and then mod operation with **100000** is done to get the new start.

Your task is to find the minimum steps in which **end** can be achieved starting from **start**. If it is not possible to reach **end**, then return **-1**.

Example 1:

Input:

arr[] = {2, 5, 7}

start = 3, end = 30

Output:

2

Explanation:

Step 1: $3 * 2 = 6 \ \% \ 100000 = 6$

We will use dijkstra here, we will take a priority queue which will store {product, steps} and distance array will store min steps for a product.

```

4 public:
5 int minimumMultiplications(vector<int>& arr, int start, int end) {
6     // code here
7     if(start==end)return 0;
8     int n = arr.size();
9     priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>pq;
10    pq.push({0,start});
11    vector<int>dis(100000,INT_MAX);
12    dis[start] = 0;
13    while(!pq.empty()){
14        int node = pq.top().second;
15        int steps = pq.top().first;
16        pq.pop();
17        if(node==end)return steps;
18        if(node>end || steps>dis[node])continue; // we cannot do node>end then
19        //continue because numbers are being wrapped around due to modulo 1e5;
20        for(int i = 0; i<n; i++){
21            int adjstep = steps + 1;
22            int adjproduct = (node*arr[i])%100000;
23            if(dis[adjproduct]>adjstep){
24                dis[adjproduct] = adjstep;
25                pq.push({dis[adjproduct],adjproduct});
26            }
27        }
28    }
29    return -1;
30 }

```

Ques-

1976. Number of Ways to Arrive at Destination

Solved 

Medium

Topics

Companies

Hint

You are in a city that consists of n intersections numbered from 0 to $n - 1$ with **bi-directional** roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections u_i and v_i that takes $time_i$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the **shortest amount of time**.

Return the **number of ways** you can arrive at your destination in the **shortest amount of time**. Since the answer may be large, return it **modulo** $10^9 + 7$.

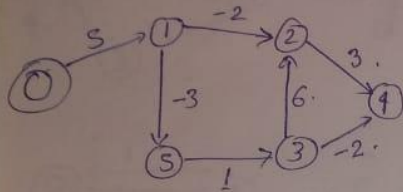
In the priority queue we will store {time,node} and in the distance array we will store {time,number of ways}

```
3  const int MOD = 1e9 + 7;
4  int countPaths(int n, vector<vector<int>>& roads) {
5      vector<vector<pair<int,int>>>adj(n);
6      for(int i = 0; i<roads.size(); i++){
7          adj[roads[i][0]].push_back({roads[i][1],roads[i][2]});
8          adj[roads[i][1]].push_back({roads[i][0],roads[i][2]});
9      }
10     priority_queue<pair<long,long>,vector<pair<long,long>>,greater<pair<long,long>>>pq;
11     pq.push({0,0}); // {time,node}
12     vector<pair<long,long>>dis(n,{LONG_MAX,0});
13     dis[0] = {0,1}; // {time,number_of_ways};
14     while(!pq.empty()){
15         int node = pq.top().second;
16         long time = pq.top().first;
17         long num_of_ways = dis[node].second;
18         pq.pop();
19         if(dis[node].first<time)continue;
20         for(auto it: adj[node]){
21             long adjnode = it.first;
22             long adjtime = it.second;
23             if(dis[adjnode].first>time+adjtime){
24                 dis[adjnode].first = time+adjtime;
25                 dis[adjnode].second = num_of_ways ;
26                 pq.push({dis[adjnode].first,adjnode});
27             }
28             else if(dis[adjnode].first == time + adjtime){
29                 dis[adjnode].second = (dis[adjnode].second + num_of_ways)%MOD;
30             }
31         }
32     }
33     return dis[n-1].second;
```


Bellman Ford Algorithm

Bellman-Ford Algorithm → helps to detect negative cycle.
 ↳ Applicable only on directed graph.

→ helps to find shortest path from single source



(0, 1, 5)
 (1, 2, -2)
 (2, 4, 3)
 (3, 2, 6)
 (3, 4, -2)
 (5, 3, 1)

→ Relax all the edges $N-1$ times sequentially.

↳ if $(dis[u] + wt < dis[v])$
 $dis[v] = dis[u] + wt$

$dis[] = [0, \infty, \infty, \infty, \infty, \infty]$

source marked as 0, every thing as ∞ .
 as from dist it is ∞ so haven't reached yet.

u	v	w
(3, 2, 6)	→	$dis[3] + 6 < dis[2]$
(5, 3, 1)	→	$dis[5] + 1 < dis[3]$
(0, 1, 5)	→	$dis[0] + 5 < dis[1] \Rightarrow 0 + 5 < dis[1]$ ↳ update 1 with 5 in $dis[]$.
(1, 5, -3)	→	$dis[1] + (-3) < dis[5] \Rightarrow 5 - 3 < dis[5]$
(1, 2, -2)	→	$dis[1] + (-2) < dis[2] \Rightarrow 5 - 2 < dis[2]$
(3, 4, -2)	→	$dis[3] - 2 < dis[4]$ (not visited)
(2, 4, 3)	→	$dis[2] + 3 < dis[4] \Rightarrow 3 + 3 < dis[4]$

like this we have to iterate 5 times.

$$\text{dis}[] = \begin{array}{|c|c|c|c|c|c|} \hline 0 & 5 & 3 & \infty & 6 & 2 \\ \hline 0 & 1 & 2 & 3 & 4 & 5 \\ \hline \end{array}$$

$$(3, 2, 6) \rightarrow 3 + 6 \overset{\text{not visited}}{<} \text{dis}[2] + 6 < \text{dis}[2]$$

$$(5, 3, 1) \rightarrow \text{dis}[5] + 1 < \text{dis}[3] \Rightarrow 3 < \text{dis}[3]$$

$$(0, 1, 5) \rightarrow \text{dis}[0] + 5 < \text{dis}[1] \Rightarrow 0 + 5 < \text{dis}[1]$$

$$(1, 5, -3) \rightarrow \text{dis}[1] - 3 < \text{dis}[5] = 5 - 3 < \text{dis}[5] \Rightarrow \text{dis}[5] \text{ as } 2$$

$$(1, 2, -2) \rightarrow \text{dis}[1] - 2 < \text{dis}[2] \Rightarrow \text{dis}[2] = 9$$

$$(3, 4, -2) \rightarrow \text{dis}[3] - 2 < \text{dis}[4] \Rightarrow \text{dis}[4] = 1$$

$$(2, 4, 3) \rightarrow \text{dis}[2] + 3 < \text{dis}[4] \Rightarrow \text{dis}[4] = 4 \text{ (not better than existing)}$$

Similarity iterate 3 more times.

Ques. Why N-1 iteration?

अगर पहले iteration में केवल $\text{dis}[0]$ मिलता है तो और distance-matrix का sequence खेला है कि एक iteration में $\text{dis}[i]$ update हो जाता है, तो maximum N-1 iteration में द्वारा update हो जाएगा एक दूसरे में affect की भी count हो जायेगी।

Ques. How it detects Negative cycle.

On Nth ^{iteration} cycle (as only N-1 iteration is done but still!) if $\text{dis}[]$ gets update then Negative cycle exists.

```

5 vector<int> bellmanFord(int V, vector<vector<int>>& edges, int src) {
6     // Code here
7     vector<int> dis(V, 1e8);
8     dis[src] = 0;
9     int n = edges.size();
10    for(int i = 0; i < V-1; i++){
11        for(int j = 0; j < n; j++){
12            int u = edges[j][0];
13            int v = edges[j][1];
14            int wt = edges[j][2];
15            if(dis[u] != 1e8 && dis[v] > dis[u] + wt){
16                dis[v] = dis[u] + wt;
17            }
18        }
19    }
20    for(int i = 0; i < n; i++){
21        int u = edges[i][0];
22        int v = edges[i][1];
23        int wt = edges[i][2];
24        if(dis[u] != 1e8 && dis[v] > dis[u] + wt){
25            return {-1};
26        }
27    }
28    return dis;
29 }
30 };
31 };

```

Floyd Warshall's Algorithm - Dijkstra and Bellmanford was single-source path algorithm while floyd warshall's is multisource path algorithm.

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

Solved 

Medium

 Topics

 Companies

 Hint

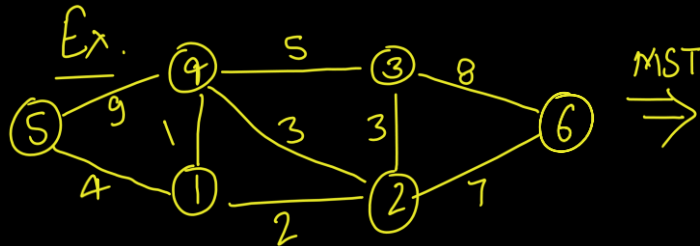
There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

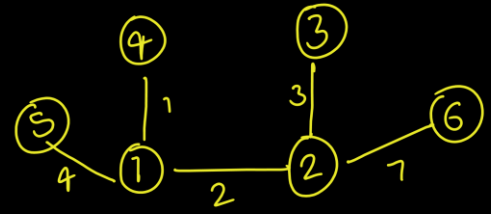
Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

```
3      int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
4          vector<vector<int>>dis(n,vector<int>(n,INT_MAX));
5          int m = edges.size();
6          for(int i = 0; i<m; i++){
7              dis[edges[i][0]][edges[i][1]] = edges[i][2];
8              dis[edges[i][1]][edges[i][0]] = edges[i][2];
9          }
10         for(int i = 0; i<n; i++){
11             dis[i][i] = 0;
12         }
13         for(int k = 0; k<n; k++){
14             for(int i = 0; i<n; i++){
15                 for(int j = 0; j<n; j++){
16                     if(dis[i][k]!=INT_MAX && dis[k][j]!=INT_MAX){
17                         dis[i][j] = min(dis[i][j],dis[i][k]+dis[k][j]);
18                     }
19                 }
20             }
21         }
22         int node = INT_MIN;
23         int mincount = INT_MAX;
24         for(int i = 0; i<n; i++){
25             int count = 0;
26             for(int j = 0; j<n; j++){
27                 if(dis[i][j]<=distanceThreshold){
28                     count++;
29                 }
30             }
31             if(count<=mincount){
32                 node = i;
33                 mincount = count;
34             }
35         }
36         return node;
37     }
38 }
39 }
```

Minimum Spanning Tree - A Tree in which we have N nodes and $N-1$ edges and all nodes are reachable from each other.



MST $\Rightarrow$



Sum = 17 (minimum of all spanning trees)

So it is MST of given graph.

Two ways to find MST -

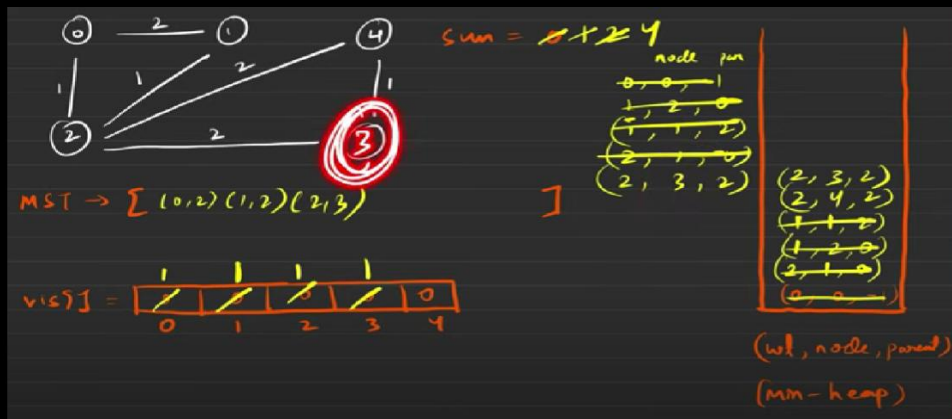
i) Prim's Algorithm.

ii) Kruskal's Algorithm. (It uses disjoint set)

i) Prim's Algorithm -

We will take priority queue, visited array. In the priority queue {dis, {node, node from which we came}}. add the value of 'dis' in the sum only when the node isn't visited. also put the {node, node from which we came} in the MST when node isn't visited.

Priority Queue will handle that only min weights are added from a node to another node, as after that we do $vis[another\ node] = 1$.



Minimum Spanning Tree

Difficulty: Medium Accuracy: 47.82% Submissions: 174K+ Points: 4 Average Time: 25m

Given a weighted, undirected, and connected graph with V vertices and E edges, your task is to find the sum of the weights of the edges in the Minimum Spanning Tree (MST) of the graph. The graph is provided as a list of edges, where each edge is represented as [u, v, w], indicating an edge between vertex u and vertex v with edge weight w.

Input: V = 3, E = 3, Edges = [[0, 1, 5], [1, 2, 3], [0, 2, 1]]

Graph visualization:

```

graph TD
    0 ---|5| 1
    1 ---|3| 2
    0 ---|1| 2
  
```

```

int spanningTree(int V, vector<vector<int>>> edges) {
    // code here
    vector<vector<pair<int,int>>>adj(V);
    for(int i = 0; i<edges.size(); i++){
        adj[edges[i][0]].push_back({edges[i][1],edges[i][2]});
        adj[edges[i][1]].push_back({edges[i][0],edges[i][2]});
    }
    vector<int>vis(V,0);
    int sum = 0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>pq;
    pq.push({0,0});
    while(!pq.empty()){
        int node = pq.top().second;
        int wt = pq.top().first;
        pq.pop();
        if(vis[node]!=1){
            vis[node] = 1;
            sum += wt;
            for(auto it: adj[node]){
                int adjnode = it.first;
                int adjwt = it.second;
                if(vis[adjnode]!=1){
                    pq.push({adjwt,adjnode});
                }
            }
        }
    }
    return sum;
}
  
```

Disjoint-Set (Union by rank, Union by Size):-

Ques Why it is used?

i) It is used in dynamic graphs.

ii) Suppose we have two components of a graph $① \rightarrow ② \rightarrow ③ \rightarrow ④$
 $⑤ \rightarrow ⑥$

now we have to find whether ① is connected to 5 or not then from DFS algo, we will take $O(N+E)$.

But Disjoint set tell the same thing in constant time.
 $O(4\alpha)$

Union by Rank-

We will take a parent array and a rank array.

- find ultimate parent of u & $u \Rightarrow pu, pv$
- find the rank of pu & pv.
- Connect smaller rank to larger rank always.

(1,2) rank = [~~0~~¹, 0, 0, ~~0~~¹, 0, ~~0~~¹, 0]

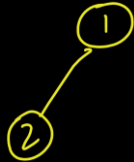
(2,3)

(4,5) parent = [~~1~~⁴, ~~2~~¹, ~~3~~¹, 4, ~~5~~⁴, ~~6~~⁴, 7]

(6,7)

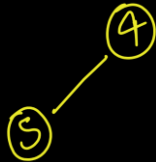
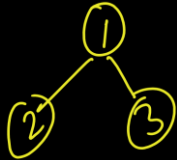
(5,6)

(3,7)



rank of ① increased by 1 as height increased.
after joining just 2 to 1, only rank of 1 increased
and parent of 2 = 1.

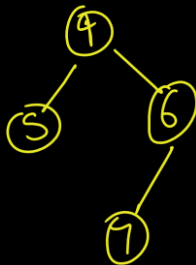
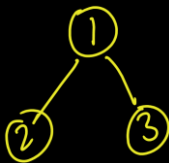
→ for (2,3)
 ultimate parent = 1
 rank = 1



So, 3 will be connected to higher rank node and ultimate parent of 2, i.e. 1.

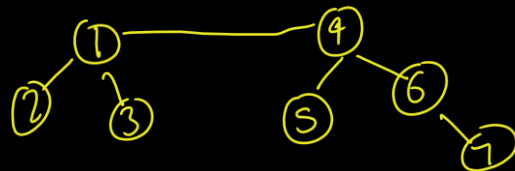
for (5,6)
 ultimate parent = 4
 rank = 1

So rank of both are equal so either 6 is connected to 4 or 4 is connected to 6.



(3,7) →
 ultimate parent = 1
 rank = 1

→ We connect ultimate parents, with lower rank ultimate parent being connected to higher rank ultimate parent.



update parent of 1 as 4

Path compression - parent of 6 is 4 & parent of 7 is 6, so update parent of 7 as 4, similarly for 2, 3.

```

3  class DisjointSet {
4      vector<int> rank, parent;
5  public:
6      DisjointSet(int n) {
7          rank.resize(n+1, 0);
8          parent.resize(n+1);
9          for(int i = 0; i <= n; i++) {
10             parent[i] = i;
11         }
12     }
13
14     int findUPar(int node) {
15         if(node == parent[node])
16             return node;
17         return parent[node] = findUPar(parent[node]);
18     }
19
20     void unionByRank(int u, int v) {
21         int ulp_u = findUPar(u);
22         int ulp_v = findUPar(v);
23         if(ulp_u == ulp_v) return;
24         if(rank[ulp_u] < rank[ulp_v]) {
25             parent[ulp_u] = ulp_v;
26         }
27         else if(rank[ulp_v] < rank[ulp_u]) {
28             parent[ulp_v] = ulp_u;
29         }
30         else {
31             parent[ulp_v] = ulp_u;
32             rank[ulp_u]++;
33         }
34     }
35 };
  
```

Union By Size:-

- ① Initially we will take an array 'size' in which initially for all indices we will keep value as 1.
- ② We will also take 'parent' array.

→ Whenever a node is attached ^{to a node(x)} increase the size corresponding to node X, by 1.

→ the ultimate parents will be attached to each other with u.p of ~~lesser~~ ^{larger} size will be attached to u.p of ~~greater~~ ^{smaller} size.

→ It is similar to union by rank. (Just name rank by size).

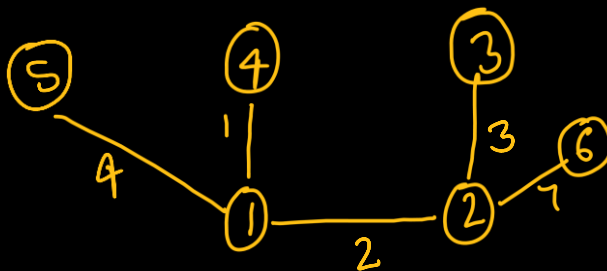
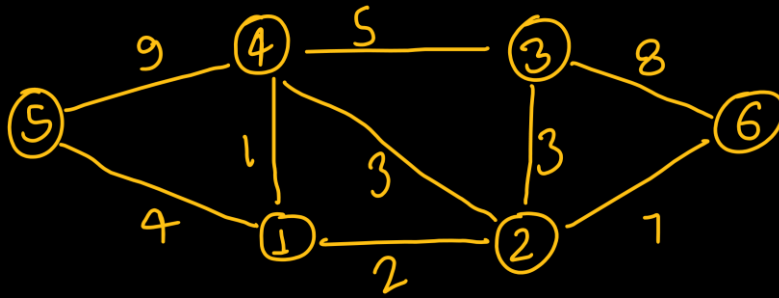
```
6 DisjointSet(int n) {
7     rank.resize(n+1, 0);
8     parent.resize(n+1);
9     size.resize(n+1);
10    for(int i = 0; i <= n; i++) {
11        parent[i] = i;
12        size[i] = 1;
13    }
14 }
15
16 int findUPar(int node) {
17     if(node == parent[node])
18         return node;
19     return parent[node] = findUPar(parent[node]);
20 }
21
22 void unionByRank(int u, int v) {
23     int ulp_u = findUPar(u);
24     int ulp_v = findUPar(v);
25     if(ulp_u == ulp_v) return;
26     if(rank[ulp_u] < rank[ulp_v]) {
27         parent[ulp_u] = ulp_v;
28     }
29     else if(rank[ulp_v] < rank[ulp_u]) {
30         parent[ulp_v] = ulp_u;
31     }
```

```
37
38 void unionBySize(int u, int v) {
39     int ulp_u = findUPar(u);
40     int ulp_v = findUPar(v);
41     if(ulp_u == ulp_v) return;
42     if(size[ulp_u] < size[ulp_v]) {
43         parent[ulp_u] = ulp_v;
44         size[ulp_v] += size[ulp_u];
45     }
46     else {
47         parent[ulp_v] = ulp_u;
48         size[ulp_u] += size[ulp_v];
49     }
50 }
```

Kruskal's Algorithm

Steps-

i) Sort all the edges according to weight.



belong to same component which is being made.

wt	u	v	
1	1	4	✓
2	1	2	✓
3	2	4	✗ (both edges belong to same component)
3	2	3	✓
4	1	5	✓
5	3	4	✗
7	2	6	✓
8	3	6	✗
9	4	5	

$$wt = 1 + 2 + 3 + 4 + 7 = 17$$

Note. We will decide whether they are from same component or not by using disjoint set.

```

2 class Disjointset{
3 private:
4     vector<int>Uparent,rank;
5 public:
6     Disjointset(int n){
7         Uparent.resize(n+1);
8         rank.resize(n+1,0);
9         for(int i = 0; i<=n; i++){
10             Uparent[i] = i;
11         }
12     }
13     int findUparent(int node){
14         if(node==Uparent[node])return node;
15         else return Uparent[node] = findUparent(Uparent[node]);
16     }
17     void unionbyrank(int u,int v){
18         int ulti_parU = findUparent(u);
19         int ulti_parV = findUparent(v);
20         if(ulti_parU==ulti_parV)return;
21         if(rank[ulti_parU]>rank[ulti_parV]){
22             Uparent[ulti_parV] = ulti_parU;
23         }
24         else if(rank[ulti_parU]<rank[ulti_parV]){
25             Uparent[ulti_parU] = ulti_parV;
26         }
27         else{
28             Uparent[ulti_parU] = ulti_parV;
29             rank[ulti_parV]++;
30         }
31     }

```

```

32 }
33 class Solution {
34 public:
35     int kruskalsMST(int V, vector<vector<int>> &edges) {
36         // code here
37         int n = edges.size();
38         vector<pair<int,pair<int,int>>>v;
39         for(int i = 0; i<n; i++){
40             v.push_back({edges[i][2],{edges[i][0],edges[i][1]}});
41         }
42         Disjointset dis(V);
43         sort(v.begin(), v.end());
44         int mst_wt = 0;
45         for(int i = 0; i<v.size(); i++){
46             if(dis.findUparent(v[i].second.first) != dis.findUparent(v[i].second.second)){
47                 mst_wt += v[i].first;
48                 dis.unionbyrank(v[i].second.first,v[i].second.second);
49             }
50         }
51         return mst_wt;
52     }
53 };

```



547. Number of Provinces

Solved

Medium

Topics

Companies

There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b , and city b is connected directly with city c , then city a is connected indirectly with city c .

A **province** is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an $n \times n$ matrix `isConnected` where `isConnected[i][j] = 1` if the i^{th} city and the j^{th} city are directly connected, and `isConnected[i][j] = 0` otherwise.

Return the total number of **provinces**.

```
1 class Disjointset{
2 public:
3     vector<int>parent,rank;
4     Disjointset(int n){
5         parent.resize(n);
6         rank.resize(n,0);
7         for(int i = 0; i<n; i++){
8             parent[i] = i;
9         }
10    }
11    int findUpar(int node){
12        if(node==parent[node])return node;
13        else return parent[node] = findUpar(parent[node]);
14    }
15    void rankbyvalue(int U , int V){
16        int parentU = findUpar(U);
17        int parentV = findUpar(V);
18        if(parentU==parentV)return;
19        if(rank[parentU]>rank[parentV]){
20            parent[parentV] = parentU;
21        }
22        else if(rank[parentU]<rank[parentV]){
23            parent[parentU] = parentV;
24        }
25        else{
26            parent[parentU] = parentV;
27            rank[parentV]++;
28        }
29    }
```

```

31 class Solution {
32 public:
33     int findCircleNum(vector<vector<int>>& isConnected) {
34         int n = isConnected.size();
35         Disjointset ds(n);
36         for(int i = 0; i<n; i++){
37             for(int j = 0; j<n; j++){
38                 if(isConnected[i][j]==1){
39                     ds.rankbyvalue(i,j);
40                 }
41             }
42         }
43         int count = 0;
44         for(int i = 0; i<n; i++){
45             if(ds.parent[i]==i){
46                 count++;
47             }
48         }
49         return count;
50     }
}

```

Ques

- You are given a graph with **n** vertices (0 to n-1) and **m** edges. You can remove **one** edge from anywhere and add that edge between **any** two vertices in **one** operation. Find the **minimum** number of operations that will be required to connect the graph. If it is not possible to connect the graph, return -1.

Example 1:

Input:

n = 4

m = 3

Edges = [[0, 1] , [0, 2] , [1, 2]]

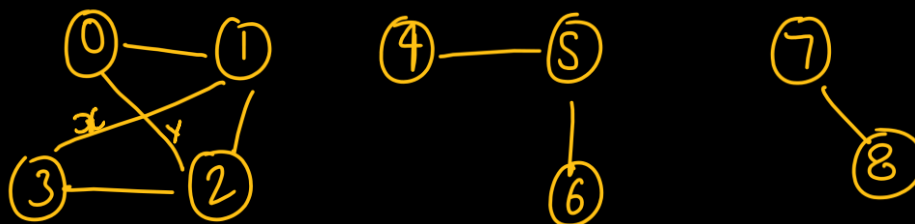
Output:

1

Explanation:

Remove edge between vertices 1 and 2 and add between vertices 1 and 3.

Ex.



here we can remove edge x & y then can use the same edge to connect 2-4 & 6-7.

→ For n vertices there should be ^{at least} $n-1$ total edges present in these graphs to get connected.

→ So, if we have $n-1$ edges then for t ultimate parents we can connect by using $t-1$ from those edges.

→ If total edges are less than $n-1$ then we can not connect all.

Connecting the graph



Difficulty: Medium

Accuracy: 52.79%

Submissions: 24K+

Points: 4

Average Time: 30m

You are given a graph with n vertices (0 to $n-1$) and m edges. You can remove **one** edge from anywhere and add that edge between **any** two vertices in **one** operation. Find the **minimum** number of operations that will be required to connect the graph. If it is not possible to connect the graph, return -1.

Example 1:

Input:

$n = 4$

$m = 3$

Edges = [[0, 1] , [0, 2] , [1, 2]]

Output:

1

Explanation:

Remove edge between vertices 1 and 2 and add between vertices 1 and 3.

```
2 class Disjointset{
3 public:
4     vector<int> parent, rank;
5     Disjointset(int n){
6         parent.resize(n);
7         rank.resize(n, 0);
8         for(int i = 0; i < n; i++){
9             parent[i] = i;
10        }
11    }
12    int findUp(int node){
13        if(parent[node] == node) return node;
14        else return parent[node] = findUp(parent[node]);
15    }
16    void unionbyrank(int U, int V){
17        int parU = findUp(U);
18        int parV = findUp(V);
19        if(parU == parV) return;
20        if(rank[parU] < rank[parV]){
21            parent[parU] = parV;
22        }
23        else if(rank[parU] > rank[parV]){
24            parent[parV] = parU;
25        }
26        else{
27            parent[parU] = parV;
28            rank[parV]++;
29        }
30    }
31 }
```

```

32 class Solution {
33     public:
34         int Solve(int n, vector<vector<int>>& edge) {
35             int m = edge.size();
36             if(m<n-1)return -1;
37             Disjointset ds(n);
38             for(int i = 0; i<m; i++){
39                 ds.unionbyrank(edge[i][0],edge[i][1]);
40             }
41             int count = 0;
42             for(int i = 0; i<ds.parent.size(); i++){
43                 if(ds.parent[i]==i){
44                     count++;
45                 }
46             }
47             return count-1;
48         }
49     };
50 };

```

Ques -

721. Accounts Merge

Solved

Medium Topics Companies Hint

Given a list of `accounts` where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are **emails** representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails **in sorted order**. The accounts themselves can be returned in **any order**.

Example 1:

Input: `accounts = [{"John","johnsmith@mail.com","john_newyork@mail.com"}, {"John","johnsmith@mail.com","john00@mail.com"}, {"Mary","mary@mail.com"}, {"John","johnnybravo@mail.com"}]`

Output: `[{"John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"}, {"Mary","mary@mail.com"}, {"John","johnnybravo@mail.com"}]`

2 → In the map john will be represented as 0.

Corresponding to each email create a map, mapping email to name → here for name we will use integer → the index in account array.

```

34     vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
35         map<string,int>mpp;
36         int n = accounts.size();
37         Disjointset ds(n);
38         for(int i = 0; i<n; i++){
39             for(int j = 1; j<accounts[i].size(); j++){
40                 if(mpp.find(accounts[i][j])==mpp.end()){
41                     mpp[accounts[i][j]] = i;
42                 }
43                 else{
44                     int U = mpp[accounts[i][j]];
45                     ds.unionbyrank(i,U);
46                 }
47             }
48         }
49         vector<vector<string>>merged_mails(n);
50         for(auto it: mpp){
51             string mail = it.first;
52             int node = it.second;
53             int uparent = ds.findUpar(node);
54             merged_mails[uparent].push_back(mail);
55         }
56         vector<vector<string>>ans;
57         for(int i = 0; i<n; i++){
58             if(merged_mails[i].size()!=0){
59                 vector<string>temp;
60                 temp.push_back(accounts[i][0]);
61                 sort(merged_mails[i].begin(), merged_mails[i].end());
62                 for(int j =0; j<merged_mails[i].size(); j++){
63                     temp.push_back(merged_mails[i][j]);
64                 }

```

```

65                 ans.push_back(temp);
66             }
67         }
68         return ans;
69     }
70 };

```

827. Making A Large Island

Solved ✓

Hard

Topics

Companies

You are given an $n \times n$ binary matrix `grid`. You are allowed to change at most one 0 to be 1.

Return the size of the largest **island** in `grid` after applying this operation.

An **island** is a 4-directionally connected group of 1s.

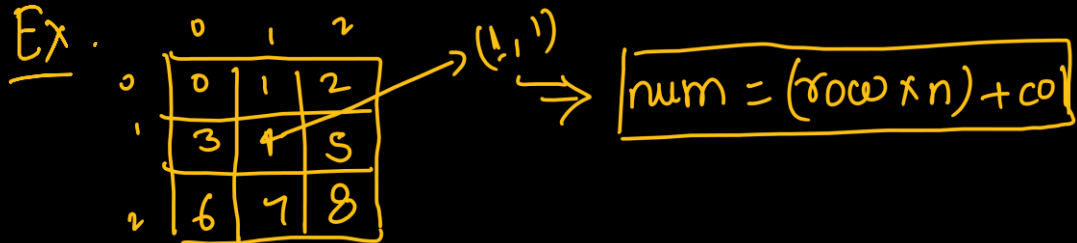
Example 1:

Input: `grid = [[1,0],[0,1]]`

Output: 3

Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.

Soln. To use disjoint set we will first convert the matrix cells into numbers.



Now, for each 1, we will first find the ultimate parent of each.

Then traverse again, go to each '0' and if its adjacent are 1. then we will find the ultimate parent of that adjacent '1', then we will check the size of ultimate parent.

We will store the ultimate parent for each zero traversing in a set, so that weight of same parent may not get counted through diff adj 1's.

	0	1	2	3	4
0	1	1	0	1	1
1	1	1	0	1	1
2	1	1	0	1	1
3	0	0	1	1	0
4	0	0	1	1	1
5	0	0	1	1	1

→ If we don't store the ult. par of this 1 in set then, we will count the size of same ultimate parent through this also.

```

15 > void unionbySize(int U, int V){...
27     }
28 };
29 class Solution {
30 private:
31     bool isValid(int newrow,int newcol,int n){
32         return newrow>=0 && newrow<n && newcol>=0 && newcol<n;
33     }
34 public:
35     int largestIsland(vector<vector<int>>& grid) {
36         int n = grid.size();
37         Disjointset ds(n*n);
38         for(int row = 0; row<n; row++){
39             for(int col = 0; col<n; col++){
40                 if(grid[row][col]==0)continue;
41                 int dr[] = {-1,1,0,0};
42                 int dc[] = {0,0,1,-1};
43                 for(int i = 0; i<4; i++){
44                     int newrow = row + dr[i];
45                     int newcol = col + dc[i];
46                     if(isValid(newrow,newcol,n) && grid[newrow][newcol]==1){
47                         int U = row*n + col;
48                         int V = newrow*n + newcol;
49                         ds.unionbySize(U,V);
50                     }
51                 }
52             }
53         }

```

```

55     int maxi = INT_MIN;
56     for(int i = 0; i < n; i++){
57         for(int j = 0; j < n; j++){
58             if(grid[i][j] == 0){
59                 set<int> st;
60                 int dr[] = {-1, 1, 0, 0};
61                 int dc[] = {0, 0, -1, 1};
62                 int size = 0;
63                 for(int d = 0; d < 4; d++){
64                     int adjrow = i + dr[d];
65                     int adjcol = j + dc[d];
66                     if(isvalid(adjrow, adjcol, n) && grid[adjrow][adjcol] == 1){
67                         int node = adjrow * n + adjcol;
68                         int Ultimate_parent = ds.findUpPar(node);
69                         if(st.find(Ultimate_parent) == st.end()){
70                             size += ds.size[Ultimate_parent];
71                             st.insert(Ultimate_parent);
72                         }
73                     }
74                 }
75                 size += 1;
76                 maxi = max(maxi, size);
77             }
78         }
79     }
80     for(int node = 0; node < n * n; node++){
81         maxi = max(maxi, ds.size[ds.findUpPar(node)]);
82     }
83     return maxi;

```

Ques-

947. Most Stones Removed with Same Row or Column

Medium

Topics

Companies

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either **the same row or the same column** as another stone that has not been removed.

Given an array `stones` of length n where `stones[i] = [xi, yi]` represents the location of the i^{th} stone, return *the largest possible number of stones that can be removed*.

Example 1:

Input: `stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]`

Output: 5

	0	1	2	3
0	✓			
1			✓	
2	✓			✓
3			✓	

To keep only one stone in a row/col we will have to remove these 3 stones.

we will use row no. as nodes & col no also as nodes.

0 1 2 3 4 5 6 7

if, there is a stone at (i, j) find union by size of (i, j)

no. of stones to be removed = total no. of stones - total no of components.

(as from each component there will be 1-stone which don't need to be deleted (as we need only one stone in a row/col.)).

one component

	0	1	2	3
0	✓	✓	✓	✓
1				✓
2				
3	✓		✓	
4				✓

	0	1	2	3	4
0	✓				
1		✓			
2			✓	✓	✓
3			✓		



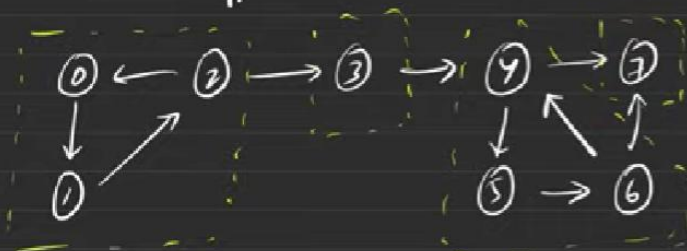

```

29 class Solution {
30 public:
31     int removeStones(vector<vector<int>>& stones) {
32         int n = stones.size();
33         int m = 0;
34         for(int i = 0; i < n; i++){
35             m = max({m, stones[i][0], stones[i][1]});
36         }
37         Disjointset ds(2*m+2);
38         for(int i = 0; i < n; i++){
39             int U = stones[i][0];
40             int V = stones[i][1]+m+1;
41             ds.unionbysize(U, V);
42         }
43         set<int> st;
44         for(int i = 0; i < n; i++){
45             st.insert(ds.findUpPar(stones[i][0]));
46             st.insert(ds.findUpPar(stones[i][1]+m+1));
47         }
48         return n-st.size();
49     }

```

(Strongly Connected Components (SCC) Kosaraju's Algorithms)

Strongly Connected Components → Kosaraju's Algorithm.

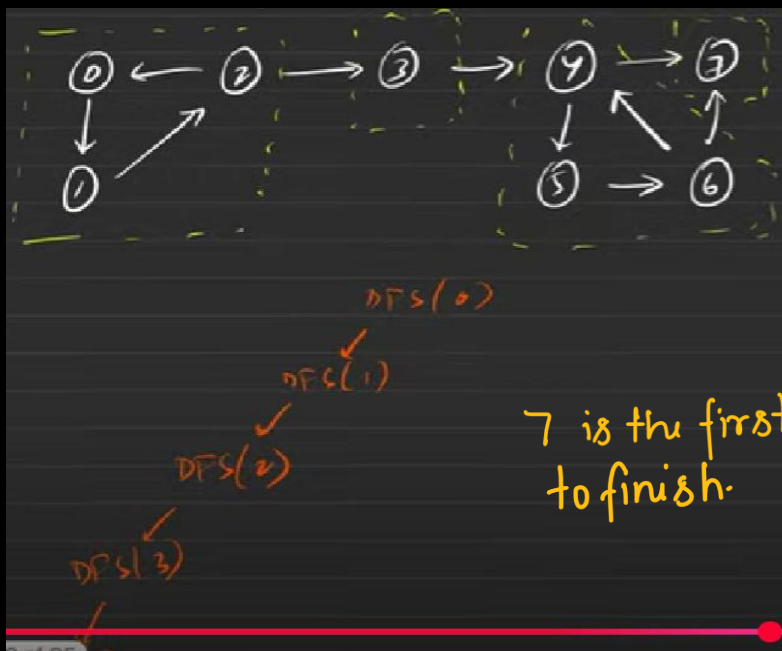


SCCs are -
 {0, 1, 2}
 {3}, {4, 5, 6}
 {7}

SCC means we can move between any pair of nodes.

Algorithm

- 1) Sort all the edges according to finishing time in dfs.
- 2) Reverse the graph.
- 3) Do a DFS.



Strongly Connected



Difficulty: **Medium**

Accuracy: **50.61%**

Submissions: **120K+**

Points: **4**

Average Time: **20m**

Given an adjacency list, **adj** of **Directed Graph**, Find the number of strongly connected components in the graph.

```

2 class Solution {
3 private:
4     void dfs(int node, vector<vector<int>> &adj, vector<int>&vis, stack<int>&st){
5         vis[node]=1;
6         for(auto it : adj[node]){
7             if(vis[it]!=1){
8                 dfs(it,adj,vis,st);
9             }
10        }
11        st.push(node);
12    }
13    void dfs3(int node, vector<vector<int>>&adj, vector<int>&vis){
14        vis[node]=1;
15        for(auto it : adj[node]){
16            if(vis[it]!=1){
17                dfs3(it,adj,vis);
18            }
19        }
20    }

```

```

22 public:
23     int kosaraju(vector<vector<int>> &adj) {
24         // code here
25         int n = adj.size();
26         vector<int> vis(n, 0);
27         stack<int> st;
28         for(int i = 0; i < n; i++){
29             if(!vis[i]){
30                 dfs(i, adj, vis, st);
31             }
32         }
33         vector<vector<int>> adj2(n);
34         for(int i = 0; i < n; i++){
35             for(int j = 0; j < adj[i].size(); j++){
36                 adj2[adj[i][j]].push_back(i);
37             }
38         }
39         int scc = 0;
40         vector<int> vis2(n, 0);
41         while(!st.empty()){
42             int node = st.top();
43             st.pop();
44             if(!vis2[node]){
45                 scc++;
46                 dfs3(node, adj2, vis2);
47             }
48         }
49         return scc;
50     }

```

→ also used in regular dfs but in kosaraju, it will be used.

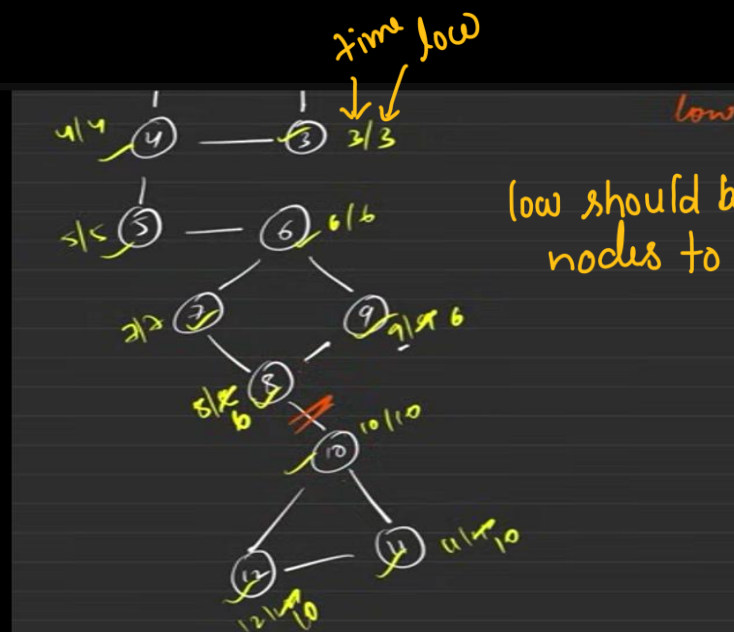
Bridges in a graph:-

→ An edge between two nodes is called bridge if removing it divides the graph into two or more components.

We will use 2 arrays -

time[] → to store at which step the node is called for the dfs.

low[] → min lowest time of insertion of all adjacent nodes apart from the parent node.



low should be different for adjacent nodes to be bridge.

1192. Critical Connections in a Network

Solved

Hard

Topics

Companies

Hint

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server `connections` forming a network where `connections[i] = [ai, bi]` represents a connection between servers `ai` and `bi`. Any server can reach other servers directly or indirectly through the network.

A *critical connection* is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

```

2 private:
3 int timer = 1;
4 void dfs(int parent,int node,vector<vector<int>>&adj,vector<vector<int>>&bridge,
vector<int>&vis,int low[], int time[]){
5     vis[node] = 1;
6     time[node] = low[node] = timer;
7     timer++;
8     for(auto it: adj[node]){
9         if(it==parent)continue;
10        if(vis[it]==0){
11            dfs(node,it,adj,bridge,vis,low,time);
12            low[node] = min(low[node],low[it]);
13            if(low[it]>time[node]){
14                bridge.push_back({it,node});
15            }
16        }
17        else{
18            low[node] = min(low[node],low[it]);
19        }
20    }
21 }

```

```

22 public:
23 vector<vector<int>> criticalConnections(int n, vector<vector<int>>& connections) {
24     vector<vector<int>>adj(n);
25     for(int i = 0; i<connections.size(); i++){
26         adj[connections[i][0]].push_back(connections[i][1]);
27         adj[connections[i][1]].push_back(connections[i][0]);
28     }
29     vector<int>vis(n,0);
30     int low[n];
31     int time[n];
32     vector<vector<int>>bridges;
33     dfs(-1,0,adj,bridges,vis,low,time);
34     return bridges;
35 }
36 };

```

Articulation points in a graph - Node on whose removal graph breaks down into multiple components.

We will use

Arrays here one is time[] and other is low[],

Time[]-> the time of insertion of the nodes in the dfs

Low[]-> minimum of all adjacent nodes time of insertion apart from parent and visited.

here $low[it] \geq time[node]$ to have articulation point.
 && parent != -1.

If a adjacent node is visited then don't take its low[] value, rather take its time value while calculating low[it].

A starting point will be articulation-point only if it has multiple children.

Articulation Point - I

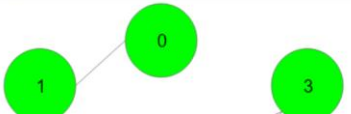
Difficulty: Hard Accuracy: 39.26% Submissions: 86K+ Points: 8 Average Time: 20m

Given an undirected connected graph with V vertices and adjacency list adj . You are required to find all the vertices removing which (and edges through it) disconnects the graph into 2 or more components and return it in sorted manner.

Note: Indexing is zero-based i.e nodes numbering from (0 to $V-1$). There might be loops present in the graph.

Example 1:

Input:



```
3- class Solution {
4- private:
5-     int timer = 1;
6-     void dfs(int node, int parent, vector<int>&vis, int time[], int low[], vector<int>&adj) {
7-         vis[node] = 1;
8-         time[node] = low[node] = timer++;
9-         int child = 0;
10-        for(auto it: adj[node]){
11-            if(it==parent)continue;
12-            if(!vis[it]){
13-                dfs(it, node, vis, time, low, mark, adj);
14-                low[node] = min(low[node], low[it]);
15-                if(low[it]>=time[node] && parent!=-1){
16-                    mark[node] = 1;
17-                }
18-            }
19-            child++;
20-        }
21-        else{
22-            low[node] = min(low[node], time[it]); // for visited it.
23-        }
24-    }
25-    if(child>1 && parent!=-1){
26-        mark[node] = 1;
27-    }
28- }
```

```
29 public:
30     vector<int> articulationPoints(int n, vector<int> adj[]) {
31         // Code here
32         vector<int>vis(n,0);
33         int time[n];
34         int low[n];
35         vector<int>mark(n,0); // will mark the articulation points;
36         for(int i = 0; i<n; i++){
37             if(!vis[i]){
38                 dfs(i, -1, vis, time, low, mark, adj);
39             }
40         }
41         vector<int>ans;
42         for(int i = 0; i<n; i++){
43             if(mark[i]==1){
44                 ans.push_back(i);
45             }
46         }
47         if(ans.size()==0)return {-1};
48         return ans;
49     }
50 }
51 }
```